

PRÁCTICA 3

Implementación del Protocolo I2C en ESP32-C6

Interfaces de HW y SW

MicroPython

1. Objetivo

Diseñar e implementar un sistema embebido basado en un microcontrolador ESP32-C6 capaz de adquirir señales analógicas provenientes de sensores, procesarlas mediante una clasificación binaria basada en umbrales predefinidos y controlar actuadores físicos, incorporando mecanismos de visualización de información mediante una pantalla LCD y estrategias de ahorro energético mediante modos de bajo consumo.

1.1 Objetivos Específicos

- Configurar y utilizar los canales ADC del ESP32-C6 para adquirir señales analógicas provenientes de sensores de temperatura.
- Implementar un algoritmo de comparación por umbrales que permita clasificar las señales de entrada en estados lógicos para la toma de decisiones.
- Controlar un foco y un ventilador mediante salidas digitales, activándolos o desactivándolos de acuerdo con las condiciones de temperatura detectadas.
- Establecer comunicación I²C entre el ESP32-C6 y una pantalla LCD para visualizar información relevante del sistema, como el valor de temperatura y el estado de los actuadores.
- Implementar un botón de control que permita habilitar o deshabilitar el modo de ahorro de energía del sistema.
- Aplicar técnicas de reducción de consumo energético utilizando modos de suspensión (Deep Sleep), manteniendo la supervisión periódica de las variables críticas del sistema.
- Integrar y utilizar las librerías necesarias para el manejo de periféricos, incluyendo la pantalla LCD y dispositivos de señalización visual.
- Validar el funcionamiento del sistema mediante pruebas de adquisición, procesamiento y control, verificando la correcta respuesta de los actuadores ante diferentes condiciones de temperatura.

1.2 Alcance de la Práctica

La presente práctica comprende el desarrollo de un sistema de control de temperatura de tipo binario (ON/OFF) implementado sobre una plataforma basada en el microcontrolador ESP32-C6. El sistema adquiere la señal analógica de un sensor de temperatura mediante los convertidores analógico-digitales (ADC) integrados en la placa de desarrollo, permitiendo la evaluación de la variable medida respecto a umbrales previamente establecidos.

A partir de esta evaluación, se generan señales de control digitales para el accionamiento de actuadores encargados de modificar la temperatura del sistema, específicamente un foco y un ventilador. La etapa de control y la etapa de potencia se encuentran eléctricamente aisladas mediante optoacopladores y relevadores, garantizando la protección del microcontrolador frente a las cargas de potencia.

La práctica incluye la adquisición de señales analógicas, el procesamiento de datos mediante lógica de decisión ON/OFF, el acondicionamiento de señales de control para el accionamiento de actuadores, la visualización de información en una pantalla LCD mediante comunicación I²C y la implementación de mecanismos básicos de ahorro energético mediante modos de suspensión del microcontrolador.

Quedan fuera del alcance de esta práctica los sistemas de control continuo o proporcional, algoritmos de control avanzados (PID, lógica difusa o control adaptativo), el acondicionamiento analógico complejo de señales, la comunicación inalámbrica y el almacenamiento permanente de datos. El propósito se limita a demostrar la integración hardware-software para la adquisición, procesamiento y control de una variable física mediante una estrategia de control discreta de dos estados.

2. Hardware Utilizado

2.1 Componentes

Componente	Cantidad	Descripción
ESP32-C6	1	Microcontrolador principal con soporte I2C, WiFi y BLE
Pantalla LCD 16x2 con módulo I2C (PCF8574)	1	Salida visual del sistema
Push Button	2	Entradas de usuario (BTN1 y BTN2)
Resistencias 4.7 kΩ	2	Pull-up externas para líneas I2C (SDA/SCL)
Foco 127VAC	1	Actuador utilizado para incrementar la temperatura del entorno de prueba.
LM35	1	Sensor de temperatura analógico cuya salida de voltaje es proporcional a la temperatura medida.
Optoacoplador	2	Elemento de aislamiento eléctrico entre la etapa de control y la etapa de potencia.
Relevador (Relay)	2	Dispositivo electromecánico utilizado para conmutar las cargas de potencia.
Ventilador	1	Actuador utilizado para disminuir la temperatura del entorno de prueba.

2.2 Mapa de Pines

Variable	GPIO ESP32-C6	Resistencia Interna	Función
btn2	GPIO 4	PULL_DOWN	Activa el modo de energía.
SCL (I2C)	GPIO 7	4.7 kΩ externas	Señal de reloj I2C
SDA (I2C)	GPIO 6	4.7 kΩ externas	Datos I2C
Foco	GPIO 3	Sin especificación	Aumento de temperatura.
Ventilador	GPIO 2	Sin especificación	Disminución de temperatura.

2.3 Esquema de Conexión

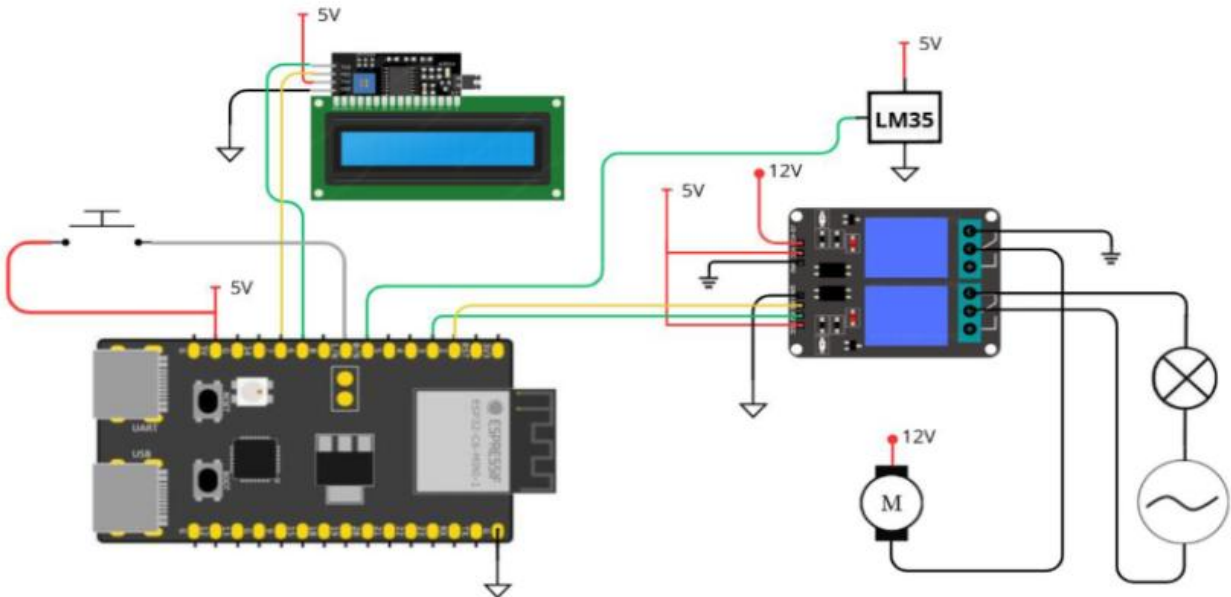


Figura 1. Esquema de conexión.

2.4 Pruebas Eléctricas Realizadas

Antes de cargar el firmware se realizaron las siguientes verificaciones eléctricas para garantizar la integridad del circuito, el correcto funcionamiento de los dispositivos de entrada y salida, y prevenir daños al microcontrolador:

- **Alimentación del sensor LM35:** Se verificó la correcta alimentación del sensor de temperatura y se midió su voltaje de salida a temperatura ambiente. La señal obtenida se encontró dentro del rango esperado para el dispositivo, confirmando su correcto funcionamiento antes de realizar la adquisición mediante el ADC.

- **Verificación de entradas analógicas (ADC):** Se midió el voltaje presente en los canales ADC utilizados para la práctica, comprobando que las señales permanecieran dentro del rango admitido por el convertidor analógico-digital del ESP32-C6. Esto garantiza lecturas válidas y evita saturaciones o errores de conversión.
- **Comunicación I²C con la pantalla LCD:** Se verificó la continuidad de las líneas SDA y SCL entre el ESP32-C6 y el módulo I²C de la pantalla LCD. Adicionalmente, se comprobó la presencia de las resistencias pull-up correspondientes para asegurar una comunicación confiable entre ambos dispositivos.
- **Circuito de accionamiento mediante optoacopladores:** Se verificó la polaridad y continuidad de las conexiones de los optoacopladores, confirmando el aislamiento eléctrico entre la etapa de control (ESP32-C6) y la etapa de potencia. Esto permite proteger al microcontrolador frente a transitorios y perturbaciones generadas por las cargas.
- **Funcionamiento de los relevadores:** Se comprobó la continuidad de los contactos normalmente abiertos (NO) y normalmente cerrados (NC), verificando que el cambio de estado ocurriera correctamente al energizar la bobina del relevador.

3. Arquitectura del Software

El proyecto está organizado en tres archivos Python que separan responsabilidades de manera clara:

Archivo	Responsabilidad
p2.py	Lógica principal: GPIO, interrupciones, timer, bucle de visualización y modo sleep.
i2c_lcd.py	Driver de bajo nivel para la LCD: inicialización del controlador HD44780 vía I2C y escritura de bytes nibble por nibble.
lcd_api.py	Capa de abstracción (API) de la LCD: comandos estándar (clear, home, move_to, putstr) independientes del hardware.

Sin tomar en cuenta librerías estándar de Micropython.

4. Descripción Técnica Detallada

El programa fue desarrollado en MicroPython para el microcontrolador ESP32-C6. Su función principal consiste en adquirir la temperatura mediante un sensor analógico, calcular un valor promedio para reducir el ruido de medición, ejecutar una estrategia de control ON/OFF para accionar un foco y un ventilador, mostrar información en una pantalla LCD mediante comunicación I²C e implementar un modo de ahorro energético utilizando suspensión temporal del procesador.

4.1 inclusión de librerías

En esta etapa se importan las librerías necesarias para acceder a los periféricos del microcontrolador y a los módulos externos.

```

from machine import ADC, Pin, SoftI2C
from time import sleep
from lib.i2c_lcd import I2cLcd
import math

```

4.2 Adquisición de temperatura

Cada constante representa un comando o máscara de bits que se envía directamente al controlador de la LCD:

```

lectura = adc.read()
voltaje = (lectura * VREF) / ADC_MAX
temperatura = ((voltaje * 1000) / 10)

```

4.3 Filtrado mediante promedio

La función promedio() reduce el efecto del ruido presente en las mediciones. Se almacenan 100 muestras consecutivas. Posteriormente, se calcula la media aritmética.

```

def promedio():
    tamaño = 100
    global vector
    if len(vector) == tamaño:
        prom = sum(vector) / tamaño
        vector = []
        return prom, True
    else:
        return 0, False

```

4.4 Visualización de información

La función Mostrar() actualiza la pantalla LCD. Muestra la temperatura promedio calculada. Muestra el estado de los actuadores, de esta forma el usuario puede conocer en tiempo real la condición del sistema.

```

def Mostrar(temp_promedio):
    lcd.clear()
    lcd.move_to(0, 0)
    lcd.putstr("Temp: {:.2f}".format(temp_promedio) + chr(223) + "C")

    # Estado del foco
    FOCO = Foco.value()
    VENTILADOR = Ventilador.value()

    lcd.move_to(0, 1)
    if FOCO == 1 and VENTILADOR == 0:
        lcd.putstr("Foco: ENCENDIDO ")
    elif FOCO == 0 and VENTILADOR == 1:
        lcd.putstr("Vent: ENCENDIDO ")
    elif FOCO == 1 and VENTILADOR == 1:
        lcd.putstr("Ambos encendidos")
    else:
        lcd.putstr("Todo apagado ")

```

4.5 Inicialización del sistema

Antes de entrar al ciclo principal se inicializan los periféricos. Posteriormente se definen los parámetros de control. Siendo el margen la banda de histéresis.

```
# PROGRAMA PRINCIPAL
CONFIG_ADC()
CONFIG_LCD()

margen = 2
temp_deseada = 35
```

4.6 Algoritmo de control ON/OFF

La estrategia implementada corresponde a un control binario con banda muerta. Donde a través de condiciones, se define que actuador es el que se enciende.

```
# ---- Lógica de control ----
if temp_promedio < temp_deseada - (margen / 2):
    Foco.on()
    Ventilador.off()
elif temp_promedio > temp_deseada + (margen / 2):
    Foco.off()
    Ventilador.on()
else:
    # Dentro del margen, no cambia estado
    pass
```

5. Flujo General del Sistema

El siguiente diagrama describe el flujo de ejecución completo del firmware desde el arranque:

```
INICIO → Importar módulos → CONFIG_ADC() → CONFIG_LCD()
      ↓
BUCLE PRINCIPAL
├─ Leer temperatura
├─ Calcular promedio
├─ [listo == True]
│   ├── [promedio < T_deseada] → Foco prendido → Ventilador apagado
│   ├── [promedio > T_deseada] → Foco apagado → Ventilador prendido
│   ├── Mostrar temperatura → Mostrar estados de actuadores
│   └─ [Estado == True] → Modo de ahorro
INTERRUPCIONES (paralelas al bucle):
  BTN  (GPIO1 ↓) → cambiar_estado() → Estado = not Estado
  Lightsleep(s) → Despierta del modo de ahorro pasado (s)
```

6. Conclusiones

En la presente práctica se logró implementar un sistema de monitoreo y control de temperatura, integrando tanto elementos de hardware como de software. Mediante el uso del convertidor analógico-digital (ADC) fue posible adquirir la señal proveniente del sensor de temperatura y procesarla para obtener una estimación de la temperatura del sistema en tiempo real.

A partir de las mediciones realizadas, se implementó una estrategia de control binario ON/OFF que permitió accionar un foco y un ventilador en función de una temperatura de referencia y una banda de histéresis definida. Esta estrategia demostró ser adecuada para aplicaciones donde no se requiere un control continuo de la variable, ofreciendo una solución simple y efectiva para el control térmico.

7. Referencias

[4] LM35, LM35 Precision Centigrade Temperature Sensors Datasheet, Texas Instruments, 2023. [Texas Instruments - LM35 Datasheet](#)

8. Anexos

```
from machine import ADC, Pin, SoftI2C
from time import sleep
from lib.i2c_lcd import I2cLcd
import math

# CONFIG LCD
def CONFIG_LCD():
    global lcd
    I2C_ADDR = 0x27
    I2C_NUM_ROWS = 2
    I2C_NUM_COLS = 16
    i2c = SoftI2C(sda=Pin(7), scl=Pin(6), freq=400000)
    lcd = I2cLcd(i2c, I2C_ADDR, I2C_NUM_ROWS, I2C_NUM_COLS)

# VARIABLES GLOBALES
estado = False
vector = []
Foco = Pin(3, Pin.OUT)
Ventilador = Pin(2, Pin.OUT)
tiempo_sleep_ms = 5000 # tiempo de sleep en milisegundos (5 segundos)
```



```

# INTERRUPTCION EXTERNA
def cambiar_estado(pin):
    global estado
    estado = not estado
    print("Estado cambiado:", estado)

BtnEstado = Pin(1, Pin.IN, Pin.PULL_DOWN)
BtnEstado.irq(handler=cambiar_estado, trigger=Pin.IRQ_FALLING)

# CONFIG ADC
def CONFIG_ADC():
    global adc
    adc = ADC(Pin(0))
    adc.atten(ADC.ATTN_0DB)      # rango ~0 - 1.1 V
    adc.width(ADC.WIDTH_12BIT)   # resolución 0 - 4095

def leer_temperatura():
    VREF = 1.1
    ADC_MAX = 4095
    lectura = adc.read()
    voltaje = (lectura * VREF) / ADC_MAX
    temperatura = ((voltaje * 1000) / 10)  # ejemplo: 10 mV/°C
    return temperatura, voltaje, lectura

def promedio():
    tamano = 100
    global vector
    if len(vector) == tamano:
        prom = sum(vector) / tamano
        vector = []
        return prom, True
    else:
        return 0, False

def Mostrar(temp_promedio):
    lcd.clear()
    lcd.move_to(0, 0)
    lcd.putstr("Temp: {:.2f}".format(temp_promedio) + chr(223) + "C")

    # Estado del foco
    FOCO = Foco.value()
    VENTILADOR = Ventilador.value()

    lcd.move_to(0, 1)
    if FOCO == 1 and VENTILADOR == 0:
        lcd.putstr("Foco: ENCENDIDO ")
    elif FOCO == 0 and VENTILADOR == 1:
        lcd.putstr("Vent: ENCENDIDO ")
    elif FOCO == 1 and VENTILADOR == 1:

```

```

        lcd.putstr("Ambos encendidos")
    else:
        lcd.putstr("Todo apagado  ")

# PROGRAMA PRINCIPAL
CONFIG_ADC()
CONFIG_LCD()

margen = 2
temp_deseada = 35

while True:
    temp, volt, lec = leer_temperatura()
    vector.append(temp)
    temp_promedio, listo = promedio()

    if listo:
        # ---- Lógica de control ----
        if temp_promedio < temp_deseada - (margen / 2):
            Foco.on()
            Ventilador.off()
        elif temp_promedio > temp_deseada + (margen / 2):
            Foco.off()
            Ventilador.on()
        else:
            # Dentro del margen, no cambia estado
            pass

        # ---- Mostrar en consola y LCD ----
        print("Temp promedio: {:.2f} °C".format(temp_promedio))
        print("Foco:", Foco.value(), "| Ventilador:", Ventilador.value())
        Mostrar(temp_promedio)

    if estado:
        # ---- Entra en modo ahorro ----
        print("Entrando en modo sleep por", tiempo_sleep_ms / 1000, "s...")
        lightsleep(tiempo_sleep_ms)
        # Al terminar el sleep, se despierta y repite el ciclo

    sleep(0.01)

```